

BrightLearn Exercise 2 SQL, Aggregates and Operators

① The Students Table:

Table students			
student_id	name	age	department
1	Alice	20	IT
2	Bob	22	HR
3	Charlie	21	IT
4	Diana	23	Finance
5	Eve	22	HR

- i) List all distinct departments in the students table
- ```
SELECT DISTINCT department
FROM students;
```

| department |
|------------|
| IT         |
| HR         |
| Finance    |

Check Columns:

- i) department ✓

2) Get the average age of students per department :  
 SELECT department, AVG(age) AS avg-age  
 FROM students  
 GROUP BY department;

| department | ave age |
|------------|---------|
| IT         | 20.5    |
| HR         | 22      |
| Finance    | 23      |

Check Columns:

- I) department ✓
- II) avg-age ✓

3) Show departments with more than 1 student :  
 SELECT department, COUNT(\*) AS student\_count  
 FROM Students  
 GROUP BY department  
 HAVING COUNT(\*) > 1;

| department | Student count |
|------------|---------------|
| IT         | 2             |
| HR         | 2             |

Check Columns:

- I) department ✓
- II) student\_count ✓



4) Get all students whose age is between 21 and 23:

```
SELECT student_id, name, age, department
FROM students
WHERE age BETWEEN 21 AND 23;
```

| student_id | name    | age | department |
|------------|---------|-----|------------|
| 2          | Bob     | 22  | HR         |
| 3          | Charlie | 21  | IT         |
| 4          | Diana   | 23  | Finance    |
| 5          | Eve     | 22  | HR         |

Check Columns:

- I) student\_id
- II) name
- III) age
- IV) department

5) List all students in the IT or HR department who are older than 21:

```
SELECT student_id, name, age, department.
FROM students
WHERE department IN ('IT', 'HR')
AND age > 21;
```

| Student_id | name | age | department |
|------------|------|-----|------------|
| 2          | Bob  | 22  | HR         |
| 5          | Eve  | 22  | HR         |

Check Columns:

student\_id, name, age, department ✓

② The Courses Table:

| course_id | course_name  | department | credits |
|-----------|--------------|------------|---------|
| 101       | SQL Basics   | IT         | 3       |
| 102       | Python       | IT         | 4       |
| 103       | Data Science | IT         | 4       |
| 104       | Excel        | Finance    | 2       |
| 105       | Statistics   | HR         | 3       |

6) ~~SQL~~ Show total credits per department, only for departments with more than 5 total credits

```

SELECT department, SUM(credits) AS total_credits
FROM courses-table
GROUP BY department
HAVING SUM(credits) > 5 ;

```

| department | total_credits |
|------------|---------------|
| IT         | 11            |

Check Columns:

- I) department. ✓
- II) total\_credits. ✓



7) List all courses that do not have 4 credits :

```
SELECT course_id, course_name, department, credits
FROM courses
WHERE credits < 4;
```

| course_id | course_name | department | credits |
|-----------|-------------|------------|---------|
| 101       | SQL Basics  | IT         | 3       |
| 104       | Excel       | Finance    | 2       |
| 105       | Statistics  | HR         | 3       |

Check Columns:

course\_id, course\_name, department, credits. ✓

8) Show the top 3 Courses by credits in descending order :

```
SELECT course_id, course_name, credits
FROM courses
ORDER BY credits DESC
LIMIT 3;
```

| course_id | course_name  | credits |
|-----------|--------------|---------|
| 102       | Python       | 4       |
| 103       | Data Science | 4       |
| 101       | SQL Basics   | 3       |

Check Columns:

1) course\_id, course\_name, credits.

③ The Enrollments Table:

| enrollment_id | student_id | course_id | grade |
|---------------|------------|-----------|-------|
| 1             | 1          | 101       | 85    |
| 2             | 2          | 102       | 78    |
| 3             | 3          | 103       | 90    |
| 4             | 4          | 104       | 88    |
| 5             | 5          | 105       | 82    |

9) Get the maximum, minimum, and average grade across all enrollments:

```
SELECT
 MAX(grade) AS max_grade,
 MIN(grade) AS min_grade,
 AVG(grade) AS avg_grade
FROM enrollment;
```

| max_grade | min_grade | avg_grade |
|-----------|-----------|-----------|
| 90        | 78        | 84.6      |

Check Columns:

- max\_grade ✓
- min\_grade ✓
- avg\_grade ✓



10) Count how many enrollments exist per course:

```
SELECT course-id, COUNT(*) AS enrollment-count
FROM enrollments
GROUP BY course-id;
```

| course-id | enrollment-count |
|-----------|------------------|
| 101       | 1                |
| 102       | 1                |
| 103       | 1                |
| 104       | 1                |
| 105       | 1                |

Check Columns:

- course-id, enrollment-count ✓

④ The Salaries Table:

| employee-id | name  | department | salary | bonus |
|-------------|-------|------------|--------|-------|
| 1           | Tom   | IT         | 60 000 | 5000  |
| 2           | Jerry | HR         | 55 000 | 4000  |
| 3           | Spike | Finance    | 70 000 | 6000  |
| 4           | Tyson | IT         | 62 000 | 5500  |
| 5           | Butch | HR         | 54 000 | 3500  |

11) Find total salary and total bonus per department:

```
SELECT
 department,
 sum(salary) AS total_salary,
 sum(bonus) AS total_bonus
FROM salaries
GROUP BY department;
```

| department | total_salary | total_bonus |
|------------|--------------|-------------|
| IT         | 122 000      | 10 500      |
| HR         | 109 000      | 7 500       |
| Finance    | 70 000       | 6 000       |

Check Columns:

- department, total\_salary, total\_bonus

12) Show departments where average salary is above 55 000:

```
SELECT
 department,
 AVG(salary) AS avg_salary
FROM salaries
GROUP BY department
HAVING AVG(salary) > 55 000;
```

| department | avg_salary |
|------------|------------|
| IT         | 61 000     |
| Finance    | 70 000     |

Check Columns: department, avg\_salary



- 13) List employees whose salary plus bonus is greater than 60 000

```
SELECT
 employee-id,
 name,
 salary,
 bonus,
 (salary + bonus) AS total-compensation
FROM salaries
WHERE (salary + bonus) > 60 000;
```

| employee-id | name  | salary | bonus | total-compensation |
|-------------|-------|--------|-------|--------------------|
| 1           | Tom   | 60 000 | 5000  | 65 000             |
| 2           | Jerry | 55 000 | 4000  | 59 000             |
| 3           | Spike | 70 000 | 6000  | 76 000             |
| 4           | Tyke  | 62 000 | 5500  | 67 500             |

Check Columns:

- employee-id, name, salary, bonus, total-compensation. ✓

⑤ The Projects Table:

| project-id | project-name   | department | budget  |
|------------|----------------|------------|---------|
| 1          | AI App         | IT         | 120 000 |
| 2          | Payroll System | Finance    | 80 000  |
| 3          | Dashboard      | IT         | 150 000 |
| 4          | Website        | Marketing  | 60 000  |
| 5          | HR Portal      | HR         | 50 000  |

- 14) Show total and average budget per department.  
Only include departments with average budget above 70,000:

```
SELECT
 department,
 SUM(budget) AS total-budget,
 AVG(budget) AS avg-budget
FROM projects
GROUP BY department
HAVING AVG(budget) > 70000;
```

| department | total-budget | avg-budget |
|------------|--------------|------------|
| IT         | 270 000      | 135000     |
| Finance    | 80 000       | 80000      |

Check Columns:

- department, total-budget, avg-budget

- 15) List all projects with budgets between 50,000 and 120,000, excluding the Marketing department:

```
SELECT
 project-id,
 project-name,
 department,
 budget
FROM projects
WHERE budget BETWEEN 50000 AND 120000
AND department <> 'Marketing';
```



| project_id | project_name   | department | budget |
|------------|----------------|------------|--------|
| 1          | AI App         | IT         | 120000 |
| 2          | Payroll System | Finance    | 80000  |
| 5          | HR Portal      | HR         | 50000  |

Check Columns:

- I) project\_id
- II) project\_name
- III) department
- IV) budget